Guía de ejecución

Cómo poner en marcha StockFlow, paso a paso

Versión para macOS

StockFlow — Inventario, compras internas y movimientos de almacén (PA-06)

Docker · PostgreSQL 16 · Spring Boot 3.5 · IntelliJ IDEA · DataGrip
React 19 · Vite 8 · WebStorm
Android Studio · Expo SDK 58 · React Native 0.88

Para quién	Cualquier integrante del equipo, sin experiencia previa en Docker, Spring Boot, React ni Android
Sistema	macOS (Apple Silicon o Intel) con Docker Desktop, Node.js y JetBrains Toolbox
Tiempo	Primera vez: 60-90 minutos (incluye instalar; la app móvil suma unos 30) · Después: menos de 5 minutos
Fecha	19 de octubre de 2026 · rama Hector (backend, web y app móvil hasta la guía móvil 03)

Los mensajes del backend y de la web se copiaron de un arranque real; en macOS son iguales salvo las rutas. La app móvil se probó en Linux: los pasos propios de macOS (rutas del SDK, imagen arm64) siguen la documentación oficial de Android y Expo, sin haberse ejecutado en una Mac.

Índice

1. Antes de empezar 3

2. Preparación (se hace una sola vez) 5
- Preparación 1 · Instalar las herramientas (una sola vez) 5
- Preparación 2 · Comprobar que todo quedó instalado 5
- Preparación 3 · Descargar el proyecto (o actualizarlo) 5

3. Poner en marcha la base y el backend 7
- Paso 1 · Encender Docker Desktop 7
- Paso 2 · Levantar la base de datos 7
- Paso 3 · Comprobar que la base está lista 8
- Paso 4 · Abrir el backend en IntelliJ IDEA 8
- Paso 5 · Elegir Java 21 en IntelliJ (sólo la primera vez) 8
- Paso 6 · Ejecutar StockFlowApplication 9
- Paso 7 · Comprobar en el navegador que el backend responde 9
- Paso 8 · Probar la API con Swagger UI 9
- Paso 9 · Probar todos los casos con requests.http 10
- Paso 10 · Ver el SQL que ejecuta Hibernate 10
- Paso 11 · Ver los datos en DataGrip 10
- Paso 12 · Ejecutar las pruebas automáticas 11

4. Poner en marcha la aplicación web (React) 12
- Web 1 · Abrir el frontend en WebStorm 12
- Web 2 · Instalar las dependencias (la primera vez) 12
- Web 3 · Revisar la dirección del backend (.env.development) 12
- Web 4 · Encender la web 13
- Web 5 · Entrar en modo demostración 13
- Web 6 · Recorrer las pantallas del menú 14
- Web 7 · Probar el CRUD desde las pantallas 14
- Web 8 · Ejecutar las pruebas, el lint y el build 15

5. Poner en marcha la app móvil (React Native + Expo) 16
- Móvil 1 · Instalar Android Studio y el Android SDK (una sola vez) 16
- Móvil 2 · Configurar ANDROID_HOME y adb (una sola vez) 16
- Móvil 3 · Crear y encender el teléfono virtual 17
- Móvil 4 · Instalar las dependencias de la app (la primera vez) 17
- Móvil 5 · Encender Metro y abrir la app en el emulador 18
- Móvil 6 · Si Expo Go no abre la app (Android 17) 18
- Móvil 7 · Recorrer las pantallas 19
- Móvil 8 · Revisar los tipos de TypeScript 19

6. Cómo apagar todo 20

7. Rutina rápida (para los días siguientes) 21

8. Problemas frecuentes y cómo resolverlos 22

9. Glosario 24

1. Antes de empezar

Esta guía te lleva, paso a paso, desde la computadora apagada hasta tener StockFlow funcionando: la base de datos encendida, el backend respondiendo, la API probada, los datos visibles en DataGrip, la aplicación web abierta en el navegador y la **app móvil** corriendo en un teléfono Android virtual.

No necesitas saber Docker, Spring Boot, React ni Android para seguirla. Cada paso dice qué hacer, el comando exacto para copiar, qué deberías ver si salió bien y qué hacer si falla. Está escrita para macOS.

Hazla en orden la primera vez: la base y el backend (capítulo 3), la web (capítulo 4) y la app móvil (capítulo 5). La web necesita al backend encendido; la app móvil, por ahora, no (ver el aviso del capítulo 5). Para el uso diario alcanza con la rutina rápida del capítulo 7.

Cómo leer cada paso

```
Los bloques oscuros son comandos: cópialos tal cual en la Terminal
```

> **✔ Qué deberías ver**
> Lo que aparece cuando el paso salió bien.

> **⚠ Si falla**
> Qué hacer si ves otra cosa.

> **i Para entenderlo**
> Por qué se hace así (opcional, pero útil para la defensa).

Las piezas del sistema

Pieza	Qué es (en simple)	Dónde vive
Docker	Un programa que ejecuta otros programas dentro de “cajas” aisladas (contenedores). Lo usamos para no instalar PostgreSQL directamente en tu sistema.	Docker Desktop, se abre como cualquier aplicación (carpeta Aplicaciones)
PostgreSQL	La base de datos: guarda categorías, productos, stock, movimientos…	Contenedor `stockflow-postgres`, en `localhost:5432`
Backend (Spring Boot)	El programa Java que recibe las peticiones, aplica las reglas del negocio y lee o escribe en PostgreSQL.	IntelliJ (`StockFlowApplication`), en `localhost:8080`
Flyway	Crea las tablas automáticamente al arrancar el backend, con los archivos V1, V2 y V3.	`src/main/resources/db/migration`
Frontend web (React + Vite)	La aplicación web: menú, tablas y formularios. No guarda datos: se los pide al backend.	`webstorm/stockflow-frontend`, con `npm run dev` en `localhost:5173`
Node.js y npm	Node.js ejecuta las herramientas de la web y del móvil (Vite, Expo, las pruebas). npm descarga sus librerías, como Maven en el backend.	Instalados en tu sistema
App móvil (React Native + Expo)	La aplicación Android. Hoy muestra datos de ejemplo guardados en la propia app; se conectará al backend en la guía móvil 08.	`androidstudio/stockflow-mobile`

Pieza	Qué es (en simple)	Dónde vive
Android Studio y el emulador	Instala el Android SDK y crea un teléfono virtual (AVD) donde corre la app. `adb` es el comando que habla con ese teléfono.	SDK en `~/Library/Android/sdk`
Metro y Expo Go	Metro empaqueta el código de la app mientras programas; Expo Go es la app del teléfono que lo descarga y lo muestra.	Metro en `localhost:8081`; Expo Go dentro del emulador
Clientes de prueba	Swagger UI, `requests.http` y Postman prueban el backend sin la web. DataGrip mira la base.	Navegador, IntelliJ, Postman y DataGrip

Cómo se conectan: Tú, en el navegador → web en `localhost:5173` → backend en `localhost:8080` → PostgreSQL en `localhost:5432` ← DataGrip. La app móvil corre en el emulador y hoy usa **sólo datos de ejemplo**: todavía no le pide nada al backend.

2. Preparación (se hace una sola vez)

Preparación 1 · Instalar las herramientas (una sola vez)

Abre la app Terminal (Cmd+Espacio → Terminal). La forma más simple es con Homebrew. La primera línea lo instala (pide tu contraseña de macOS); las otras, los programas:

```
/bin/bash -c "$(curl -fsSL \
  https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install git node
brew install --cask docker-desktop jetbrains-toolbox postman
```

Importante: al terminar, Homebrew muestra *Next steps* con uno o dos comandos (`echo … >> ~/.zprofile` y `eval …`). Ejecútalos: si no, la terminal no encuentra `brew`. Java no hace falta: IntelliJ lo descarga en el capítulo 3, paso 5. Android Studio se instala en el capítulo 5.

✔ Qué deberías ver

Cada `brew install` termina sin `Error:`. Docker, JetBrains Toolbox y Postman aparecen en Aplicaciones.

⚠ Si falla

Si `brew install --cask docker-desktop` no encuentra el programa, descarga Docker Desktop de docker.com/products/docker-desktop eligiendo *Mac with Apple chip* o *Intel chip* (menú Apple → Acerca de esta Mac).

Preparación 2 · Comprobar que todo quedó instalado

En una Terminal nueva:

```
git --version
docker --version
node -v
npm -v
```

✔ Qué deberías ver

Cuatro versiones, por ejemplo `git version 2.5x`, `Docker version 28.x`, `v22.x` y `10.x` o superiores. Node.js debe ser 20.19 o superior.

⚠ Si falla

Si alguno dice `command not found`, cierra y abre la terminal; si sigue igual, repite su instalación.

Preparación 3 · Descargar el proyecto (o actualizarlo)

Si es la primera vez en esta computadora:

```
cd ~/Desktop
git clone https://github.com/HD4nielBu/StockFlow.git
cd StockFlow
git switch Hector
```

Si ya lo tenías: `cd ~/Desktop/StockFlow` y `git pull`.

✔ Qué deberías ver

`git status -sb` muestra `## Hector...origin/Hector` sin `behind`.

⚠ Si falla

Si `git clone` pide usuario y contraseña, GitHub ya no acepta la contraseña: usa un token personal (GitHub → Settings → Developer settings → Personal access tokens) o una llave SSH, o pide acceso al dueño del repositorio.

i Para entenderlo

Si Escritorio y Documentos están en iCloud, iCloud puede sincronizar `node_modules` y `target`. Si notas lentitud, clona en `~/dev/StockFlow` y usa esa ruta en el resto de la guía.

3. Poner en marcha la base y el backend

Sigue los pasos en orden. Del 1 al 7 dejan la base y el backend funcionando; del 8 al 12 sirven para probar la API y para mostrar el proyecto en la defensa.

Paso 1 · Encender Docker Desktop

Abre Docker Desktop desde Aplicaciones (o Cmd+Espacio → Docker). La primera vez acepta las condiciones y elige *Use recommended settings*; puedes omitir el inicio de sesión (Skip). Cuando diga **Engine running**, en la Terminal:

```
docker ps
```

✔ Qué deberías ver

Una tabla con los títulos `CONTAINER ID IMAGE …` (aunque esté vacía).

⚠ Si falla

`Cannot connect to the Docker daemon at unix:///Users/…/docker.sock`: Docker Desktop está cerrado o todavía arrancando. Ábrelo y espera.

i Para entenderlo

Docker Desktop es una aplicación normal: no se enciende con comandos. Para no abrirlo cada vez: Settings → General → *Start Docker Desktop when you sign in*.

Paso 2 · Levantar la base de datos

Desde la carpeta del proyecto:

```
cd ~/Desktop/StockFlow
docker compose up -d
```

Datos de conexión (los usa el backend; tú los necesitas para DataGrip):

Dato	Valor
Host	localhost
Puerto	5432
Base de datos	stockflow (tablas en el esquema stockflow)
Usuario	stockflow_admin
Contraseña	Cambia_Esta_Clave_2026!

✔ Qué deberías ver

La primera vez descarga la imagen `postgres:16` y termina con `Container stockflow-postgres Started`. Las siguientes veces responde enseguida.

⚠ Si falla

`no configuration file provided`: no estás en la carpeta del proyecto (revisa el `cd`). `port is already allocated` en el 5432: ya tienes otro PostgreSQL (Homebrew o Postgres.app): mira el capítulo 8.

> **i Para entenderlo**
>
> `docker compose` lee `docker-compose.yml` y crea un contenedor con PostgreSQL 16, la base `stockflow` y el usuario `stockflow_admin`. `-d` lo deja en segundo plano. La contraseña es sólo de laboratorio: para usar otra, crea un `.env` (git lo ignora) con `DB_PASSWORD=tu_clave` antes del primer `up` y pon la misma variable en IntelliJ.

Paso 3 · Comprobar que la base está lista

Espera unos 10 segundos y ejecuta:

```
docker ps
```

> **✔ Qué deberías ver**
>
> Una fila `stockflow-postgres` con `Up … (healthy)` y el puerto `0.0.0.0:5432->5432/tcp`.

> **⚠ Si falla**
>
> `(health: starting)`: espera y repite. Si no aparece la fila, repite el paso 2.

> **i Para entenderlo**
>
> *healthy* significa que PostgreSQL ya acepta conexiones (lo comprueba el healthcheck del `docker-compose.yml`).

Paso 4 · Abrir el backend en IntelliJ IDEA

Muy importante: hay que abrir la carpeta `intellij/stockflow-backend` (la que tiene `pom.xml`), no la carpeta StockFlow. Si abres otra, IntelliJ no reconoce el proyecto.

JetBrains Toolbox → IntelliJ IDEA → **Open** (o, si ya está abierto, File → Open…) → `~/Desktop/StockFlow/intellij/stockflow-backend` → Open.

Si pregunta *Trust and Open Project?*, elige **Trust Project**.

> **✔ Qué deberías ver**
>
> El panel Project muestra `stockflow-backend` con `src` y `pom.xml`. Espera a que termine la barra de progreso (*Importing Maven project*, *Indexing…*).

> **⚠ Si falla**
>
> Si el panel muestra StockFlow con `docs` y `datagrip`, abriste la carpeta equivocada: File → Close Project y repite.

Paso 5 · Elegir Java 21 en IntelliJ (sólo la primera vez)

1. Menú File → Project Structure… (Cmd+;).
2. A la izquierda elige **Project**.
3. En **SDK** debe aparecer 21. Si ya está, OK y sigue al paso 6.
4. Si no: SDK → Add SDK → JDK… y elige SDK → **Download JDK…** → Version 21 → Vendor Eclipse Temurin → Download.
5. Pulsa OK.

> **✔ Qué deberías ver**
>
> Desaparece el aviso amarillo *Project JDK is not defined*.

> **⚠ Si falla**
> `invalid source release: 21` o `Unsupported class file major version`: el SDK no es el 21; repite este paso.

Paso 6 · Ejecutar StockFlowApplication

1. Pulsa Cmd+O, escribe `StockFlowApplication` y Enter.
2. A la izquierda de `public class StockFlowApplication` hay un triángulo verde ▶.
3. Clic en el triángulo → **Run 'StockFlowApplication'**.
4. Abajo se abre la ventana Run con el registro (log) del arranque.

Desde la segunda vez: el ▶ verde de la barra superior o Ctrl+R.

> **✔ Qué deberías ver**
> Las líneas importantes, copiadas de un arranque real del proyecto:
> ✓ `Tomcat initialized with port 8080 (http)` → el servidor web se preparó en el puerto 8080.
> ✓ `HikariPool-1 - Start completed.` → se conectó a PostgreSQL.
> ✓ `Database: jdbc:postgresql://localhost:5432/stockflow (PostgreSQL 16.15)` → Flyway encontró la base.
> ✓ `Schema "stockflow" is up to date. No migration necessary.` → las tablas ya existen. Con una base nueva verías `Migrating schema "stockflow" to version "1 …"`, luego 2 y 3.
> ✓ `Started StockFlowApplication in 3.186 seconds` → ¡listo!

> **⚠ Si falla**
> `APPLICATION FAILED TO START` o una excepción en rojo: busca el mensaje en el capítulo 8. Si macOS pregunta si *java* puede aceptar conexiones entrantes, puedes elegir Denegar: para usar StockFlow en esta Mac no hace falta.

> **ℹ Para entenderlo**
> No cierres esa ventana ni pulses el cuadrado rojo ■ mientras pruebas: el backend vive mientras esa ejecución esté activa.

Paso 7 · Comprobar en el navegador que el backend responde

```
http://localhost:8080/api/health
```

> **✔ Qué deberías ver**
> Un JSON como `{"status":"OK", "stage":"HEXAGONAL_RELACION_1N", "project":"StockFlow - Inventario y Movimientos de Almacén (PA-06)", "application":"stockflow-backend", …}`. Después prueba `http://localhost:8080/api/categorias` (las 4 categorías de la semilla) y `http://localhost:8080/api/productos` (los 5 productos).

> **⚠ Si falla**
> *No se puede conectar* o *Connection refused*: el backend no está corriendo; vuelve al paso 6.

Paso 8 · Probar la API con Swagger UI

```
http://localhost:8080/swagger-ui.html
```

1. Verás los grupos categoria-controller, producto-controller, ubicacion-controller e inventario-controller.

2. Abre `GET /api/productos/{id}` → **Try it out** → id = 1 → **Execute**: `200` y el JSON de *Resma papel bond A4 75g*.
3. Prueba un error: id = 9999 → `404` con *No existe el producto con id: 9999*.

> **✔ Qué deberías ver**
> Cada respuesta con su código HTTP (200, 201, 404…) y el JSON.

> **⚠ Si falla**
> Si la página no carga, revisa que el backend siga corriendo (paso 6).

Paso 9 · Probar todos los casos con requests.http

1. En IntelliJ pulsa Cmd+Shift+O, escribe `requests.http` y Enter.
2. Cada petición tiene un ▶ verde a su izquierda: la respuesta aparece abajo.
3. Ejecútalas **en orden**: los primeros POST guardan el id creado en `{{categoriaId}}` y `{{productoId}}`, que usan los PUT y DELETE.
4. El título de cada petición dice el código esperado, por ejemplo `### [409] Código UNIQUE repetido`.

Si IntelliJ pregunta por un *environment*, elige **No Environment**.

> **✔ Qué deberías ver**
> Cada respuesta con el código que indica su título: 201, 200, 400, 404, 409, 422 o 204.

> **⚠ Si falla**
> Si un PUT o DELETE con `{{productoId}}` falla, ejecuta primero el POST que lo crea. Si ejecutas todo dos veces, algunos POST darán 409 porque el código ya existe: es correcto.

Paso 10 · Ver el SQL que ejecuta Hibernate

Mientras pruebas, mira la ventana Run del paso 6: con cada petición aparecen las sentencias SQL, por ejemplo `select cje1_0.categoria_id, …`. Un POST muestra `insert into stockflow.producto …`, un PUT `update …` y un DELETE `delete from …`.

> **✔ Qué deberías ver**
> SQL con formato legible después de cada petición.

> **ℹ Para entenderlo**
> Está activado en `application.yml` (`org.hibernate.SQL: debug` y `format_sql: true`). Es la evidencia de que un POST llegó a PostgreSQL.

Paso 11 · Ver los datos en DataGrip

1. Abre DataGrip (desde Toolbox).
2. **+** → Data Source → PostgreSQL.
3. Host localhost · Port 5432 · Database stockflow · User stockflow_admin · Password Cambia_Esta_Clave_2026!
4. Si aparece *Download missing driver files*, descárgalos.
5. **Test Connection** → *Succeeded* → OK.
6. Despliega stockflow → stockflow → tables: 18 tablas (17 del proyecto + `flyway_schema_history`).
7. File → Open → `~/Desktop/StockFlow/datagrip/03_verificacion_y_pruebas.sql`; cursor en una consulta y Cmd+Enter.

✔ Qué deberías ver

Los resultados en una tabla debajo del editor.

⚠ Si falla

La sección 7 del script tiene pruebas que **deben** fallar (muestran que las constraints funcionan). La sección 8 modifica datos y es segura de repetir.

Paso 12 · Ejecutar las pruebas automáticas

1. En el panel Project, clic derecho sobre `src/test/java`.
2. **Run 'All Tests'**.

✔ Qué deberías ver

Barra verde y `Tests passed: 60` (comprobado el 19 de octubre de 2026). La primera vez tarda más porque descarga PostgreSQL para las pruebas de integración.

⚠ Si falla

Si Docker está apagado, las 11 pruebas de `PersistenciaPostgresTest` aparecen omitidas; no es un error. Enciende Docker y repite.

i Para entenderlo

Las pruebas no tocan tu base: `PersistenciaPostgresTest` crea un PostgreSQL temporal (Testcontainers) y lo borra al terminar.

4. Poner en marcha la aplicación web (React)

La web vive en `webstorm/stockflow-frontend` y se trabaja con WebStorm. Necesita la base y el backend del capítulo 3 encendidos: si no, abre, pero no muestra datos.

Vas a tener dos programas corriendo a la vez: el backend en IntelliJ (8080) y la web en la terminal de WebStorm (5173). El navegador abre el 5173 y la web le pide los datos al 8080.

Web 1 · Abrir el frontend en WebStorm

Antes: el backend tiene que estar corriendo (capítulo 3, paso 6). Hay que abrir la carpeta exacta `webstorm/stockflow-frontend`, la que tiene `package.json`.

Toolbox → WebStorm → **Open** → `~/Desktop/StockFlow/webstorm/stockflow-frontend` → Open → Trust Project.

✔ Qué deberías ver

En el panel Project: `stockflow-frontend` con `src`, `public`, `package.json`, `vite.config.ts`, `nginx.conf` y `.env.development`.

⚠ Si falla

Si ves StockFlow con `docs` e `intellij`, abriste la carpeta equivocada.

Web 2 · Instalar las dependencias (la primera vez)

Terminal de WebStorm: View → Tool Windows → Terminal (Option+F12). Ya se abre en la carpeta del frontend:

```
npm install
```

✔ Qué deberías ver

Termina con `added … packages` y aparece la carpeta `node_modules`.

⚠ Si falla

Si WebStorm muestra el aviso *Run 'npm install'*, ese botón hace lo mismo. Si aparece `ERR!` por la red, revisa tu conexión y repite.

i Para entenderlo

`node_modules` no se sube a git; cada integrante lo genera. `package-lock.json` sí está en git y fija las mismas versiones para todos. Repite `npm install` cuando un `git pull` cambie `package.json`.

Web 3 · Revisar la dirección del backend (.env.development)

Abre `.env.development` (raíz de stockflow-frontend). Debe contener:

```
VITE_API_URL=http://localhost:8080/api
```

✔ Qué deberías ver

Esa línea (más dos comentarios). No hace falta cambiar nada.

⚠ Si falla

Si el archivo no existe, créalo copiando `.env.example`.

i Para entenderlo

La web lee la dirección del backend de esta variable (`src/config/env.ts`). En producción, `.env.production` usa `/api` y `nginx.conf` la reenvía al backend. Sólo las variables `VITE_` llegan al navegador: nunca pongas contraseñas aquí.

Web 4 · Encender la web

En la terminal de WebStorm:

```
npm run dev
```

✓ Qué deberías ver

Copiado de un arranque real:

✓ `VITE v8.3.2 ready in 149 ms` → el servidor de desarrollo arrancó.

✓ `➜ Local: http://localhost:5173/` → la dirección de la aplicación.

⚠ Si falla

`Port 5173 is already in use`: ya hay otro `npm run dev`; ciérralo con Ctrl+C o búscalo con `lsof -nP -iTCP:5173 -sTCP:LISTEN`. El proyecto no se cambia de puerto a propósito: el backend sólo acepta el 5173 (CORS).

i Para entenderlo

No cierres esa terminal mientras uses la aplicación. Al guardar un archivo, la página se actualiza sola.

Web 5 · Entrar en modo demostración

Abre en el navegador:

```
http://localhost:5173/login
```

1. Verás **Entrar a StockFlow** con un aviso amarillo: *Modo demostración. StockFlow aún no tiene inicio de sesión real*.
2. Escribe cualquier correo o usuario (por ejemplo `maria.lopez@uni.edu`) y cualquier contraseña. Marca *Recordarme* si quieres que el navegador recuerde tu nombre.
3. Pulsa **Entrar en modo demo**: comprueba que el backend responde (`GET /api/health`) y te lleva al Inicio.

✓ Qué deberías ver

El **Inicio**, con el menú azul a la izquierda (Principal, Inventario, Administración):

✓ Un saludo con la fecha y cuatro accesos rápidos: Nuevo producto, Nueva categoría, Existencias y Dashboard.

✓ Indicadores con la semilla: **Productos 5**, **Categorías 4**, **Bajo el stock mínimo 1** y **Ubicaciones con stock 2**.

✓ En *Alertas de inventario*: Bolígrafo azul, Depósito Norte, **25 / 100**.

✓ Abajo a la izquierda, tu nombre y tus iniciales (*Invitado* si entraste directo a `http://localhost:5173`).

> **⚠ Si falla**
> Si al entrar aparece *El servidor no responde*, el backend está apagado: enciéndelo (capítulo 3, paso 6). El botón *Entrar sin conexión* deja pasar igual, pero las pantallas mostrarán *No se pudo conectar con el backend*.

> **ℹ Para entenderlo**
> El backend todavía no tiene usuarios ni contraseñas: este login **no es seguridad**. No verifica credenciales ni envía la contraseña; sólo guarda el nombre a mostrar en el navegador. Por eso también se puede abrir `http://localhost:5173` sin entrar.

Web 6 · Recorrer las pantallas del menú

Cada opción del menú es una ruta. Las marcadas **Demo** en el menú usan datos locales y lo dicen en pantalla; el resto viene del backend:

Menú	Qué muestra	Datos
Inicio · Dashboard	Indicadores, alertas, gráficos y últimos productos	Reales
Categorías · Productos	CRUD con búsqueda, filtros y paginación (Web 7)	Reales
Existencias	Stock por producto y ubicación, y cobertura del mínimo	Reales
Movimientos	Kardex de un producto, paginado en el servidor	Reales
Almacenes	Ubicaciones con su stock y alta de ubicaciones	Reales
Reportes	Exportaciones CSV generadas en el navegador	Reales
Usuarios · Configuración · Perfil	Roles, tema y datos de la sesión	Demo (o mixtos)

Atajo útil: el buscador de arriba (**Cmd+K**) salta a cualquier sección; el sol/luna cambia entre tema claro y oscuro.

> **✔ Qué deberías ver**
> Cada pantalla carga sin recuadros rojos. Recargar con F5 en cualquier ruta (por ejemplo `/productos`) vuelve a la misma pantalla.

> **⚠ Si falla**
> Un recuadro *No se pudieron cargar...* con **Reintentar** indica que el backend se detuvo: mira la ventana Run de IntelliJ.

Web 7 · Probar el CRUD desde las pantallas

Este recorrido demuestra que web, backend y base trabajan juntos (sirve para la defensa):

1. **Categorías → Nueva categoría**: se abre una ventana. Pulsa **Crear categoría** con todo vacío → *El código es obligatorio* y *El nombre es obligatorio* debajo de los campos. Es la validación de la web: no se envió nada.
2. Escribe código `CAT-DEMO` y nombre `Demostración` → **Crear categoría** → aviso verde *Categoría CAT-DEMO creada* y la fila aparece.
3. Repite el mismo código → dentro de la ventana, recuadro rojo *No se pudo guardar la categoría* con el texto del backend: *Ya existe una categoría con el valor: CAT-DEMO* (409).
4. En la fila *Material de oficina* abre el menú **⋯** → **Eliminar**: la confirmación ya avisa que tiene 2 productos. Si confirmas, el backend responde 409 (*...tiene productos asociados y no se puede eliminar; desactívala...*) y la fila se queda.

5. **Productos → Nuevo producto**: categoría Demostración, código `PRD-DEMO-001`, nombre `Producto demo`, stock mínimo 5 → crear. Con el menú ⋯ usa **Editar** para cambiarle el nombre y **Eliminar** para borrarlo.
6. Vuelve a Categorías y elimina CAT-DEMO (ya sin productos): desaparece.

✔ Qué deberías ver

Con cada acción, la ventana Run de IntelliJ muestra el `insert`, `update` o `delete` que llegó a PostgreSQL.

⚠ Si falla

Si algo no responde, abre la consola del navegador (Cmd+Option+I) y busca el mensaje en rojo en el capítulo 8.

i Para entenderlo

La web sólo muestra el resultado: las reglas (código único, no borrar una categoría con productos) las decide el backend. Por eso, ante un 409, la fila no se quita.

Web 8 · Ejecutar las pruebas, el lint y el build

Abre otra pestaña de la terminal (botón +) para no detener `npm run dev`:

```
npm run test:run
npm run lint
npm run build
```

✔ Qué deberías ver

Comprobado el 19 de octubre de 2026:

- ✓ `npm run test:run` → `Test Files 30 passed (30)` y `Tests 107 passed (107)`.
- ✓ `npm run lint` → sin advertencias.
- ✓ `npm run build` → `✓ built in …ms` y la carpeta `dist/`.

⚠ Si falla

`error TS…` en el build es un error de tipos: el mensaje indica archivo y línea. Las pruebas no necesitan el backend: usan datos simulados.

i Para entenderlo

Las pruebas están junto al código (`*.test.ts`, `*.test.tsx`) y usan Vitest y Testing Library.

5. Poner en marcha la app móvil (React Native + Expo)

La app vive en `androidstudio/stockflow-mobile`. Se escribe en TypeScript y corre en un teléfono Android virtual (emulador). Este capítulo cubre lo implementado hasta la guía móvil 03: pantallas de categorías y productos con navegación.

> **Importante:** hoy la app muestra **datos de ejemplo** guardados en `src/data/demoData.ts` (basados en la semilla de la base, más una categoría inactiva y otra sin productos). **No necesita el backend ni la base** y lo que veas no está en PostgreSQL. La conexión llega en la guía móvil 08 (`http://10.0.2.2:8080/api`) y el CRUD real en la 09.

Móvil 1 · Instalar Android Studio y el Android SDK (una sola vez)

Desde JetBrains Toolbox (botón Install junto a **Android Studio**) o con `brew install --cask android-studio`.

1. Ábrelo. En el asistente de primer inicio elige **Standard** y acepta las licencias: descarga el SDK.
2. Pantalla de bienvenida → **More Actions → SDK Manager**. Anota la **Android SDK Location** (normalmente `~/Library/Android/sdk`).
3. Pestaña **SDK Platforms**: deja marcada la versión que instaló el asistente y marca también **Android 16 (API 36)**, que se usará para compilar en la guía 15.
4. Pestaña **SDK Tools**: comprueba **Android SDK Platform-Tools**, **Android Emulator** y **Android SDK Build-Tools**. **Apply → OK**.

> **✔ Qué deberías ver**
> El SDK Manager muestra las piezas como *Installed* y la carpeta del SDK contiene `platform-tools`, `emulator`, `platforms` y `build-tools`.

> **⚠ Si falla**
> *Unable to access Android SDK add-on list* en el primer arranque: pulsa **Cancel** (no *Setup Proxy*) y sigue el asistente; ese aviso sólo trata de componentes opcionales. Si después no descarga nada: Settings → HTTP Proxy → *No proxy* → *Check connection* con `https://dl.google.com`.

> **i Para entenderlo**
> No se usa *New Project* de Android Studio: el proyecto ya existe y lo maneja Expo. Android Studio aporta el SDK, `adb` y el emulador.

Móvil 2 · Configurar ANDROID_HOME y adb (una sola vez)

```
# Agrega estas líneas al final de ~/.zprofile
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$ANDROID_HOME/platform-tools:$ANDROID_HOME/emulator:$PATH"
```

Después abre una Terminal nueva (o `source ~/.zprofile`).

Comprueba en una terminal nueva:

```
adb version
emulator -version
```

✔ Qué deberías ver

`Android Debug Bridge version 1.0.41` y `Android emulator version …`.

⚠ Si falla

`adb: command not found` o *no se reconoce*: la terminal se abrió antes de configurar las variables, o la ruta no coincide con la *SDK Location* del paso anterior.

i Para entenderlo

`ANDROID_HOME` le dice a Expo dónde está el SDK; el PATH permite escribir `adb` y `emulator` desde cualquier carpeta.

Móvil 3 · Crear y encender el teléfono virtual

1. Android Studio → **More Actions → Virtual Device Manager → Create Virtual Device**.
2. Categoría **Phone** → un teléfono estándar (un Pixel o *Medium Phone*, no tablet ni plegable) → Next.
3. Imagen del sistema: la más reciente estable, arm64-v8a (chip Apple) o x86_64 (Intel), con *Google Play* → Download si hace falta → Finish.
4. Pulsa ▶ junto al dispositivo y espera la pantalla de inicio de Android.

También se enciende desde la terminal (el nombre exacto lo da la primera línea):

```
emulator -list-avds
emulator -avd Medium_Phone_API_37.0
```

Con el emulador encendido:

```
adb devices -l
```

✔ Qué deberías ver

`emulator-5554 device product:sdk_gphone64_x86_64 …`: el estado debe ser **device**.

⚠ Si falla

`offline`: Android todavía está arrancando, espera. Lista vacía: el emulador no está encendido. Emulador negro o lentísimo: En una Mac con chip Apple elige una imagen **arm64-v8a**; en una Mac Intel, **x86_64**. macOS no necesita configurar nada más para acelerar el emulador.

i Para entenderlo

En el autor el asistente instaló Android 17 (API 37). Expo Go funciona en un Android más nuevo que el que usa la app para compilar.

Móvil 4 · Instalar las dependencias de la app (la primera vez)

En la Terminal:

```
cd ~/Desktop/StockFlow/androidstudio/stockflow-mobile
npm install
```

✔ Qué deberías ver

Termina con `added … packages`. Es normal que avise *vulnerabilities*: **no** ejecutes `npm audit fix --force`, cambiaría versiones que Expo necesita fijas.

⚠ Si falla

`Could not read package.json`: no estás en `stockflow-mobile`.

i Para entenderlo

Las librerías nativas se agregan siempre con `npx expo install …`, que elige la versión compatible con el SDK de Expo (58). El proyecto también se puede abrir en WebStorm o Android Studio para editar el código.

Móvil 5 · Encender Metro y abrir la app en el emulador

Con el emulador encendido, en la carpeta de la app:

```
npx expo start --android
```

✔ Qué deberías ver

Copiado de un arranque real:

✓ `Using src/app as the root directory for Expo Router.`

✓ `Starting Metro Bundler` y, la primera vez, `Fetching Expo Go → Installing Expo Go on Medium_Phone_API_37.0.`

✓ `Android Bundled 4908ms node_modules/expo-router/entry.js (1285 modules)` → la app se envió al teléfono.

En el emulador aparece **StockFlow Mobile** con **5 Categorías** y **5 Productos**.

⚠ Si falla

`Fetching Expo Go` termina en `AggregateError [ETIMEDOUT]`: la red cortó la descarga. Dale más tiempo a Node:

```
NODE_OPTIONS="--network-family-autoselection-attempt-timeout=3000" npx expo start --android
```

Si Expo Go se queda en su propia pantalla de inicio, ve al paso siguiente.

i Para entenderlo

No cierres esa terminal: es Metro. Al guardar un archivo, la app se actualiza sola (Fast Refresh). `r` recarga la app y `Ctrl+C` detiene Metro.

Móvil 6 · Si Expo Go no abre la app (Android 17)

Android 17 pide permiso para que las apps entren a la red local, y Expo Go intenta abrir la app en la IP de tu computadora. Aparece el aviso *Local network access needed* y la app no carga. En el emulador no hace falta la red: redirige el puerto de Metro y abre la app por `localhost`:

```
adb reverse tcp:8081 tcp:8081
adb shell am start -a android.intent.action.VIEW -d "exp://127.0.0.1:8081" host.exp.exponent
```

✔ Qué deberías ver

`adb reverse --list` muestra `tcp:8081 tcp:8081` y la app carga.

⚠ Si falla

Si sigue en blanco, cierra Expo Go (`adb shell am force-stop host.exp.exponent`), espera un segundo y repite la segunda línea. Con Metro detenido, nada carga: revisa el paso 5.

i Para entenderlo

`adb reverse` hace que el `localhost:8081` del teléfono llegue al `localhost:8081` de tu computadora, donde corre Metro.

Móvil 7 · Recorrer las pantallas

La app navega con Expo Router (una pantalla por archivo de `src/app`) y el botón Atrás de Android:

Acción	Qué deberías ver
Explorar categorías	Cinco categorías; Tecnología marcada **INACTIVA**
Material de oficina	Ficha con 2 productos: PRD-OFI-001 y PRD-OFI-002
Mobiliario	*Sin resultados*: una categoría puede no tener productos
Tocar PRD-OFI-001	Ficha del producto y *Ver ficha de la categoría* →
Atrás de Android	Vuelve a la pantalla anterior, en orden
Acerca de esta guía → Probar categoría 999	Mensaje de categoría no encontrada, sin cerrarse
Probar producto con ID inválido	Mensaje de ID no válido (`/productos/abc`)

✔ Qué deberías ver

Ninguna pantalla roja de error. La app avisa en el Inicio: *Datos locales de ejemplo: todavía NO existe consumo de API REST*.

⚠ Si falla

Pantalla roja: lee el archivo y la línea que indica. App vieja en pantalla: detén Metro y arráncalo con `npx expo start --clear`.

Móvil 8 · Revisar los tipos de TypeScript

En otra terminal, en la carpeta de la app:

```
npx tsc --noEmit
```

✔ Qué deberías ver

Termina sin mostrar nada (código de salida 0).

⚠ Si falla

Si se ejecuta antes del primer `npx expo start`, puede quejarse de las rutas: Expo genera sus tipos al arrancar. Arranca Metro una vez y repite.

i Para entenderlo

El avance, las pruebas y las evidencias de cada guía móvil están en `androidstudio/stockflow-mobile/docs/` .

6. Cómo apagar todo

Al terminar, apaga en este orden:

Qué apagar	Cómo	¿Se pierden datos?
La app móvil (Metro)	Ctrl+C en la terminal de `npx expo start`	No (usa datos de ejemplo)
El emulador	Cierra su ventana (o ■ en el Device Manager)	No
La web	Ctrl+C en la terminal de `npm run dev`	No (la web no guarda datos)
El backend	Cuadrado rojo ■ en la ventana Run de IntelliJ (Cmd+F2)	No
La base de datos	`cd ~/Desktop/StockFlow` y `docker compose down`	No: quedan en un volumen de Docker
Docker	Ícono de la ballena en la barra de menús → *Quit Docker Desktop*	No

Lo que NO debes hacer

- No uses `docker compose down -v` salvo que quieras borrar la base: `-v` borra el volumen; al volver a arrancar, Flyway recrea las tablas y la semilla original.
- No edites `V1__…`, `V2__…` ni `V3__…`: Flyway detecta el cambio y el backend no arranca. Los cambios de base van en un archivo nuevo (`V4__descripcion.sql`).
- No guardes contraseñas ni tokens en archivos del proyecto, y menos en un `.env` de la web (`VITE_`) o del móvil (`EXPO_PUBLIC_`): esas variables se publican en la aplicación.
- No subas `node_modules`, `dist` ni `.expo` a git y no borres los `package-lock.json`.
- No abras la web desde `http://127.0.0.1:5173` ni desde otro puerto: el backend sólo acepta `http://localhost:5173` (CORS).

7. Rutina rápida (para los días siguientes)

Cuando ya hiciste todo una vez, esto es lo único que necesitas:

Encender

1. Docker: abrir Docker Desktop.
2. `cd ~/Desktop/StockFlow` y `docker compose up -d`.
3. IntelliJ (`intellij/stockflow-backend`) → ▶ StockFlowApplication → esperar *Started StockFlowApplication*.
4. WebStorm (`webstorm/stockflow-frontend`) → terminal (Option+F12) → `npm run dev` → `http://localhost:5173/login`.
5. App móvil: encender el emulador → en `androidstudio/stockflow-mobile`, `npx expo start --android` (en Android 17, además, los dos comandos de *Móvil 6*).

Apagar

1. Ctrl+C en la terminal de Metro y cerrar el emulador.
2. Ctrl+C en la terminal de `npm run dev`.
3. ■ en IntelliJ.
4. `docker compose down`.
5. Docker: Quit Docker Desktop.

8. Problemas frecuentes y cómo resolverlos

Busca el mensaje que te aparece (en la terminal, en la ventana Run de IntelliJ, en la consola del navegador con Cmd+Option+I o en la terminal de Metro).

Lo que ves	Qué significa	Cómo solucionarlo
`Cannot connect to the Docker daemon at unix:///Users/…`	Docker Desktop cerrado o arrancando.	Ábrelo y espera *Engine running*.
`Ports are not available` en el 5432	Otro PostgreSQL (Homebrew o Postgres.app).	`brew services stop postgresql@16` (o la versión que muestre `brew services list`); en Postgres.app, Stop.
`Connection to localhost:5432 refused`	El contenedor no está corriendo.	Pasos 2 y 3 (`docker compose up -d`, esperar *healthy*).
`password authentication failed for user "stockflow_admin"`	La contraseña no coincide o el backend se conectó a otro PostgreSQL.	Detén el otro PostgreSQL. Si usas otra `DB_PASSWORD`, ponla en IntelliJ: Run → Edit Configurations… → Environment variables.
`Port 8080 was already in use`	Ya hay otro backend corriendo.	Detén la otra ejecución con ■; si no la encuentras: `lsof -nP -iTCP:8080 -sTCP:LISTEN`.
`Found non-empty schema(s) "stockflow" but no schema history table`	La base tiene tablas creadas a mano.	`docker compose down -v`, `up -d` y volver a ejecutar el backend.
`Migration checksum mismatch`	Alguien modificó una migración ya aplicada.	`git checkout -- intellij/stockflow-backend/src/main/resources/db/migration/`.
`Schema-validation: missing table/column`	Base vieja o cambio sin migración.	`git pull` en Hector; si sigue, `docker compose down -v` y `up -d`.
No aparece el ▶ verde en StockFlowApplication	IntelliJ no cargó el proyecto Maven.	Abre `intellij/stockflow-backend`; clic derecho en `pom.xml` → Maven → Reload Project.
`invalid source release: 21`	IntelliJ usa otro Java.	Paso 5 (SDK = 21).
Web: `npm: command not found` / no se reconoce	Falta Node.js o la terminal es vieja.	Instálalo (capítulo 2) y abre una terminal nueva.
Web: `sh: vite: command not found`	Falta `node_modules`.	`npm install` en `webstorm/stockflow-frontend`.
Web: `Port 5173 is already in use`	Ya hay otro `npm run dev`.	Ciérralo con Ctrl+C; si no lo encuentras: `lsof -nP -iTCP:5173 -sTCP:LISTEN`.
Web: login dice *El servidor no responde*	El backend está apagado.	Capítulo 3, paso 6; luego vuelve a pulsar *Entrar en modo demo*.
Web: recuadro *No se pudo conectar con el backend…*	El backend (8080) se detuvo.	Capítulo 3, paso 6; luego F5 o *Reintentar*.
Web, consola (Cmd+Option+I): `blocked by CORS policy`	La web se abrió desde otra dirección (127.0.0.1, otro puerto o `npm run preview`).	Usa `http://localhost:5173`.

Lo que ves	Qué significa	Cómo solucionarlo
Web: página en blanco y `Falta VITE_API_URL`	No existe `.env.development`.	Créalo desde `.env.example` y reinicia `npm run dev`.
Android Studio: *Unable to access Android SDK add-on list*	Falló la descarga de componentes opcionales en el primer arranque.	Cancel y seguir el asistente (Móvil 1).
Móvil: `adb: command not found`	Falta el PATH del SDK o la terminal es vieja.	Móvil 2 y terminal nueva.
Móvil: `adb devices` vacío u `offline`	El emulador no está encendido o arranca.	Encenderlo (Móvil 3) y esperar.
Móvil: `AggregateError [ETIMEDOUT]` al *Fetching Expo Go*	La red cortó la descarga.	Repetir con `NODE_OPTIONS` (Móvil 5).
Móvil: Expo Go muestra *Local network access needed* y no abre la app	Android 17 bloquea la IP local.	`adb reverse` y abrir `exp://127.0.0.1:8081` (Móvil 6).
Móvil: *Project is incompatible with this version of Expo Go*	Se abrió una URL o un Expo Go de otra versión.	Cerrar Expo Go y abrir de nuevo con `npx expo start --android` (instala el Expo Go correcto).
Móvil: la app muestra una versión vieja	Caché de Metro.	Ctrl+C y `npx expo start --clear`.
Móvil: emulador negro o muy lento	Falta aceleración por hardware.	`emulator -accel-check` y la nota de Móvil 3.

Si el error no está en la tabla: copia el primer mensaje en rojo (en el backend, la línea `Caused by:` suele explicar la causa) y pide ayuda al equipo con ese texto exacto.

9. Glosario

Término	Significado simple
Backend	El programa que procesa las peticiones y guarda los datos. Responde JSON.
Contenedor · Volumen	Una “caja” donde corre un programa aislado (aquí, PostgreSQL) y el disco de Docker donde guarda sus datos.
localhost · Puerto	“Esta misma computadora” y el número que identifica a un programa: 5432 PostgreSQL, 8080 backend, 5173 web, 8081 Metro.
Endpoint · JSON	Una dirección de la API (`GET /api/productos/1`) y el formato de texto de los datos.
Migración · Flyway	Archivos SQL numerados (V1, V2, V3) que crean la base; Flyway los aplica en orden.
Swagger UI	Página generada automáticamente que documenta la API y permite probarla.
Maven · npm	Los gestores de librerías del backend y de la web/móvil.
Frontend · React · Vite	La interfaz web; la librería de componentes; la herramienta que la sirve (`npm run dev`) y la empaqueta (`npm run build`).
CORS	Regla del navegador: el backend sólo acepta peticiones de `http://localhost:5173`.
Modo demostración	El login de la web sin autenticación real: no verifica credenciales.
React Native	React para apps móviles: `View` y `Text` en lugar de `div` y `p`.
Expo · Expo Go	Herramientas sobre React Native para crear, ejecutar y compilar la app; Expo Go es la app de prueba que la muestra sin compilar un APK.
Metro	El empaquetador del código de la app mientras programas (puerto 8081).
Expo Router	Navegación por archivos: cada archivo de `src/app` es una pantalla.
Android SDK · AVD · adb	Las herramientas de Android; el teléfono virtual; el comando que habla con él.
10.0.2.2	Dentro del emulador, la dirección de tu computadora (allí `localhost` es el propio teléfono). La usará la app para llegar al backend desde la guía móvil 08.
Homebrew	El instalador de programas de macOS que usa esta guía.